

Ans. 1

Operating System is a system-level software layer that controls hardware and provides service to user applications. It is like a manager b/w user programmes and machine resources. The first objective of OS is Convenience, meaning user and programmers should not worry about hardware-level complexity. The second objective is effective utilization of CPU, memory, and I/O so that throughput and responsiveness remain high.

It performs process scheduling, memory allocation, file and device handling, permission control and system security. It also offers abstraction like files, virtual memory and process APIs which make application development practical. Therefore, OS is both a control program and a resource allocator.

Ans. 2

A process is an independent executing program with separate address space and resources. If one process crashes, others are usually protected due to isolation.

A thread is a smaller execution path inside a process and shares memory and resources of that process.

Threads are lightweight because creating / switching threads require less overhead than processes.

Also, communication between threads is faster because they share address space directly. ex - in a browser, UI thread fetches data and rendering thread paints content, all in parallel under one process.

Ans. 3

CPU Scheduling decide which process from ready queue should run next. it is necessary in multiprogramming because many process are ready but only one can use CPU core at a time.

FCFS is non-preemptive and very simple. it follow arrival order, which is fair in queue sense but often inefficient.

A long job at front delay smaller jobs and increase waiting time (convey effect).

Round Robin is preemptive and assign each process a fixed quantum. This improve fairness and response time because no process can hold CPU too long. But RR has context-switch overhead, especially when quantum is too small. In practice, RR is preferred for interactive system while FCFS may suit simple batch workload.

Ans. 4

Deadlock is a condition in which processes are blocked forever because each is waiting for a resource held by another process. Coffman condition explain when deadlock can occur:

1. Mutual exclusion: resources cannot be shared
2. Hold and wait: process keep one resource while asking another.

3. No preemption : resources can not be forcefully taken.

4. Circular wait : chain of process waiting for each other

If all four happen together, deadlock is possible.

Example : P₁ hold printer and want Scanner ; P₂ hold Scanner and want printer. Neither proceed.
OS can handle deadlock through prevention, avoidance or detection / recovery method.

Ans. 5

virtual memory provide each process a large logical memory space, even when physical RAM is limited. The OS stores less active pages on disk and load required pages into RAM on demand.

Paging divide logical memory into pages and physical memory into equal-sized frames, then maps them via page table.

Main benefit are improved multiprogramming, process isolation, and reduced external fragmentation because contiguous physical allocation is not required. But if page fault happen too often, performance drops due to disk latency. So page replacement policy and frame allocation are important for efficiency.

Ans. 6

Synchronization is required when multiple thread / process access shared data concurrently. without Synchronization, race condition occur and output become non-deterministic. Critical Section is the part of code ~~with~~ where shared data is accessed & must be protected.

Mutex is lock for mutual exclusion : one thread enters, other wait. Semaphore is signaling counter and can allow one or more thread depending on binary or counting type. Semaphore are useful in producer-consumer control and resources pool. Proper Synchronization ensure correctness, consistency and safe concurrent execution.

Ans. 7

Fragmentation means wasted memory due to allocation pattern. Internal fragmentation happens when allocated block is larger than requested memory, leaving unused bytes inside that block. External fragmentation occur when free memory is scattered into small holes and large contiguous allocation fails. Internal fragmentation can be reduced by using better allocation size, while external fragmentation can be reduced using compaction or paging. Paging is especially effectively against external fragmentation, though some internal waste may still exist in the last page.